Utility based Spreadsheet

Ajinkya Dharme
2023CS10761

Akshit Abhinav Kujur
2023CS10100

Manvendra Rajpurohit
2023CS10936

April 25, 2025

1 Overview

This project implements a programmable spreadsheet application in Rust with both CLI and GUI. It **includes advanced functionality such as command history, a custom debugger mode, password-protected cells, undo/redo, locating cyclic dependency, if-else statements, for loops, taking input from other files, saving output in some other file, conditional formatting, plotting, recursive expressions to generate any formula and an itinerary maker**. The application also supports extensibility as you can set the cell value in the GUI itself and the GUI also incorporates a feature that highlights the cells the current selected cell depends on and also highlights the cells which are dependent on the current cell. The GUI also has zoom feature and a custom scroller for horizontal and vertical navigation across sheet. Built using Rust's ecosystem for parsing, concurrency, and GUI rendering, the project is designed for maintainability and extensibility.

2 Primary Data Structures

Spreadsheet State:

- `Spreadsheet`: Holds `Vec<i32>` values we had implemented 1d vector of size n*m to decrease the memory, `HashMap<String, Expr>` formulas like "A1"-¿its Expr formula, scroll row and col currently stored as usize, total rows and cols stores as usize `Vec<CellFormat>` for formatting, and privacy flags.
- `CellValue`: enum of Either a valid `i32` or an error state.
- `CellFormat`: Condition and color (e.g., `LessThan`, `Between`) for formatting rules.

GUI State:

- `SpreadsheetApp`: Struct managing GUI state.
  - `sheet`: `Arc<Mutex<Spreadsheet>>` for shared access.
  - `zoom`: `f32` for zoom level.
  - `quit_flag`, `gui_close_requested`: Atomic booleans for control.
  - `selected_cell`: `Option<(row, col)>`.
  - `editing_expr`: String for active input.
  - `recalc_manager`: `Arc<Mutex<RecalcManager>>`.
  - `precedent_cells`, `dependent_cells`: Vectors of `(row, col)`.

Commands:

- `Command`: Enum for all user actions—cell updates, formatting, privacy, scrolling, plotting, I/O, conditionals, loops, GUI toggle, and exit.

Debugger:

- `DebugSession`: Struct holding `Vec<String>` to track changes triggered by cell edits.

Dependency Management:

- `RecalcManager`: Handles formula dependencies, topological sorting, cycle detection, and range-based updates.

3 Interfaces between Software modules

Module Overview:

- `command`: Defines the `Command` enum and related types (`Expr`, `CellRef`, `BinaryOp`) representing user/system actions. Serves as the communication bridge between parser, evaluator, and recalculation modules.
- `parser`: Transforms user input into structured `Command` objects using combinator-based parsing (`nom`). Outputs are passed directly to the evaluator.
- `evaluator`: Executes parsed `Commands`, updating the spreadsheet state, triggering recalculations, and handling side effects like plotting or file I/O. Utilizes `eval_expr` to evaluate expressions.
- `sheet`: Implements the `Spreadsheet` data structure, supporting cell storage, formatting, scrolling, and privacy.
- `recalculation`: Manages formula dependencies via a directed graph, performing topological sorting to handle updates efficiently and avoid cycles.
- `main`: Entry point of the application. Initializes state, launches CLI/GUI, and routes commands between modules.

Key dependencies in `Cargo.toml`:

- `nom`: Parser combinator library for command and formula parsing.
- `rustyline`: Line editor for CLI input.
- `eframe`, `egui`: GUI framework for interactive spreadsheet editing.
- `winit`: Window handling for GUI.
- `plotters`: 2D plotting library for chart generation.
- `rodio`: Audio output for feedback (e.g., beep on errors).

4 Optimisations

- We have **handled range operations very effectively** as we found out that the bottle neck was coming due to these commands being not handled properly.
- Used 1d vector instead of 2d allocation of sheet
- Used iterative DFS and calculated topo order only once and used it if cycle detected did not repeat it

Test Case	Verdict	Marks	Memory(MB)	Time(ms)
hidden_tc2/chain/large_dep_chain.cmds	PASS	0.625	167.2109375	312.9127025604248
hidden_tc2/chain/small_dep_chain.cmds	PASS	0.625	144.80859375	31.349897384643555
hidden_tc2/dense_dag/large.cmds	PASS	1.0	156.71484375	207.13210105895996
hidden_tc2/dense_dag/small.cmds	PASS	1.0	151.7265625	82.07988739013672
hidden_tc2/range1/range_ops_small_avg.cmds	PASS	0.05	143.99609375	26.480913162231445
hidden_tc2/range1/range_ops_small_max.cmds	PASS	0.05	143.8515625	25.449514389038086
hidden_tc2/range1/range_ops_small_min.cmds	PASS	0.05	143.8515625	26.038169860839844
hidden_tc2/range1/range_ops_small_stdev.cmds	PASS	0.05	144.140625	31.30960464477539
hidden_tc2/range1/range_ops_small_sum.cmds	PASS	0.05	143.8515625	25.858640670776367
hidden_tc2/range2/range_ops_large_avg.cmds	PASS	0.25	176.6953125	259.7682476043701
hidden_tc2/range3/range_ops_small_reassign_avg.cmds	PASS	0.05	143.8515625	28.06687355041504
hidden_tc2/range3/range_ops_small_reassign_max.cmds	PASS	0.05	143.8515625	22.564411163330078
hidden_tc2/range3/range_ops_small_reassign_min.cmds	PASS	0.05	143.8515625	24.24931526184082
hidden_tc2/range3/range_ops_small_reassign_stdev.cmds	PASS	0.05	143.8515625	25.79522132873535
hidden_tc2/range3/range_ops_small_reassign_sum.cmds	PASS	0.05	143.8515625	25.992631912231445
hidden_tc2/range4/range_ops_large_reassign_avg.cmds	PASS	0.75	176.6953125	244.05717849731445

(a) tc2

Test Case	Verdict	Marks	Memory(MB)	Time(ms)
hidden_tc3/chain/large.cmds	PASS	0.625	0	147.82071113586426
hidden_tc3/chain/small.cmds	PASS	0.625	0	53.56764793395996
hidden_tc3/dense/dense_large.cmds	PASS	1.0	0	1243.7682151794434
hidden_tc3/dense/dense_small.cmds	PASS	1.0	0	196.96569442749023
hidden_tc3/range1/small_avg.cmds	PASS	0.05	0	566.8845176696777
hidden_tc3/range1/small_max.cmds	PASS	0.05	0	606.3344478607178
hidden_tc3/range1/small_min.cmds	PASS	0.05	0	590.8079147338867
hidden_tc3/range1/small_stdev.cmds	PASS	0.05	0	773.5614776611328
hidden_tc3/range1/small_sum.cmds	PASS	0.05	0	594.3014621734619
hidden_tc3/range2/large.cmds	PASS	0.25	0	1947.1642971038818
hidden_tc3/range3/small_avg.cmds	PASS	0.05	0	1129.8987865447998
hidden_tc3/range3/small_max.cmds	PASS	0.05	0	1147.9594707489014
hidden_tc3/range3/small_min.cmds	PASS	0.05	0	1158.0309867858887
hidden_tc3/range3/small_stdev.cmds	PASS	0.05	0	1330.7905197143555
hidden_tc3/range3/small_sum.cmds	PASS	0.05	0	1244.196891784668
hidden_tc3/range4/large.cmds	PASS	0.75	0	4109.3504428863525

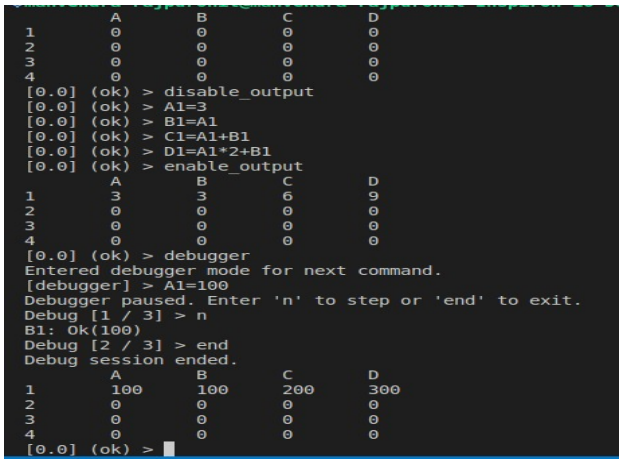
(b) tc3

Figure 1: Performance tests

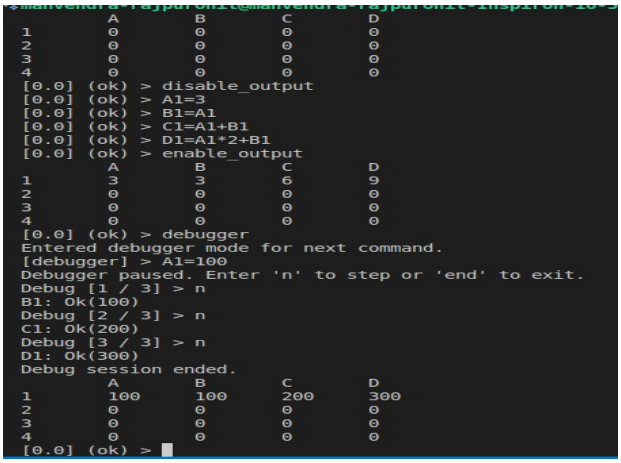
5 Extensions workthrough

5.1 Debugger mode

- Debug mode allows step-by-step inspection of recalculation, showing intermediate updates.
- When we are writing any command, many cells change their values simultaneously. For debugging purposes, we will update only one cell at a time. You can go to the next one by typing `n` and pressing Enter. Each time you do this, one cell gets updated, and its value (in the form of `Result<>`) is printed. We are doing this by updating in topological order, one cell at a time.



(a) Step 1



(b) Step 2

Figure 2: Step-by-step update view in debugger mode

5.2 Command history

- In C lab the main challenge we faced was typing the same commands again and again and if by mistake I typed a command and I want to change it I cannot move the cursor right or left. To solve this problem we made a ergonomic tool by leveraging the arrow keys.
- **uparrowkey** if you want the previous command no need to write it just press uparrow key.
- **downarrowkey** if you are on any previous command and want to go to the next command then press downarrowkey it fetches the next command if there.
- Can also use **left and right arrow key** for editing the command as in long command if I made error in writing then no need to write it again can use left and right arrow key to navigate in the command itself.

5.3 Undo/Redo

- `StateManager` maintains undo/redo stacks for spreadsheet state, allowing users to revert or reapply changes.
- Press **u** to undo.
- Press **y** to redo.

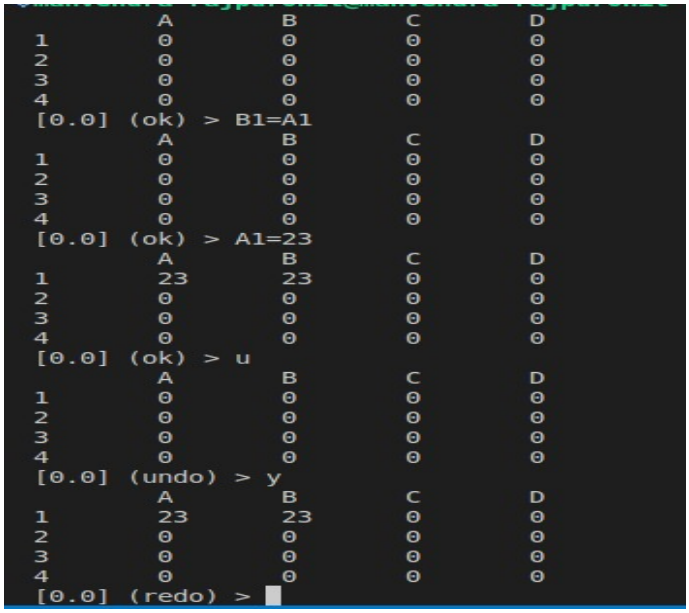


Figure 3: Undo/Redo functionality

5.4 Write any type of formulas

Leveraging the use of AST we can write any length formulas mixing different types

- A1=MAX(B1:B10)*(Sqrt(16))+B1+D1*E1
- B2=34
- A1=(sum(B1:B10)*max(B1:B10))+3*4+sleep(3)
- **advanced functions also supported** like
- A1=sqrt(8)
- A1=abs(-3)
- A1=tan(60)
- A1=sin(85)
- J10=7&1

5.5 for loop

- In some cases we need to write set of similar commands like A1=B1+1 A2=B2+2 etc we can simply write it in one single command
  - :i in 1..10: Ai = Bi + 1
  - :i in 1..10: Bi = Ci + 1

5.6 if_then_else

- if $A1 < 2$ then F1=MAX(A1:A10) else F1=10*10
- This is not a permanent command means not stored in formula its just used to update the value once otherwise we would be just complicating our languages only and hard to debug then

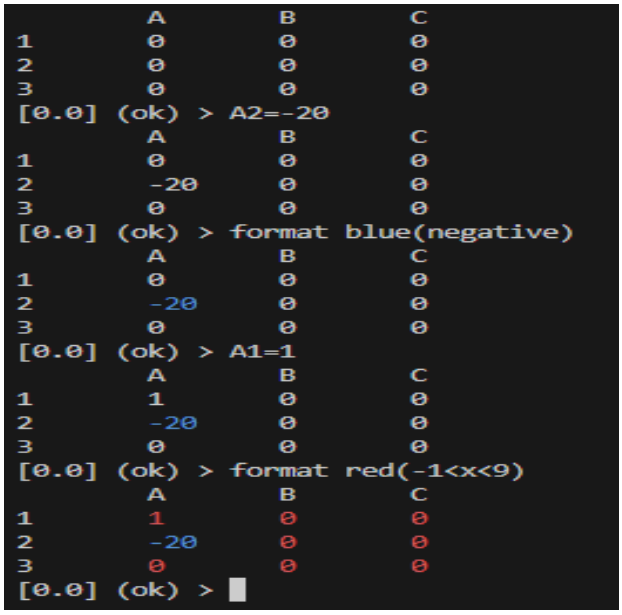
5.7 Cycle printing

- We are printing cycle as it helps in debugging and various purposes whenever a cycle is detected we print it.
  - A1=B1
  - B1=C1
  - C1=A1

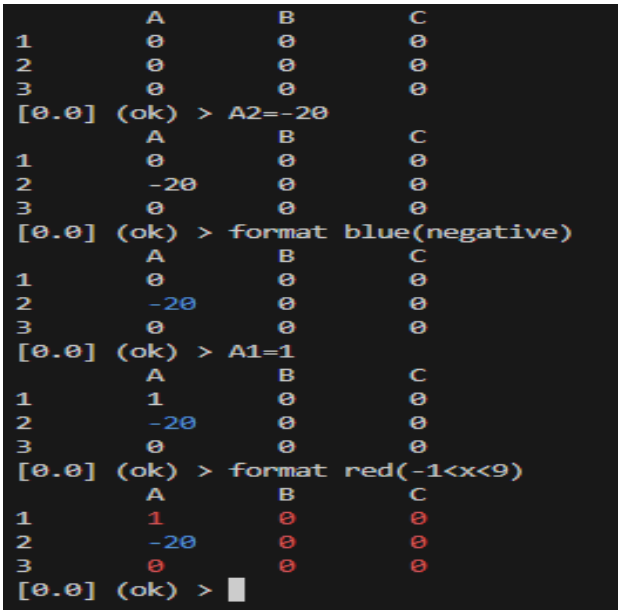
  prints:: `B1 → A1 → C1 → B1`

5.8 Conditional formatting

- Formatting of cells based on cell values
- Here left value is inclusive and right value is exclusive and formatting stays until it is removed if you apply formatting in a range than you can remove some part of the range or complete range or multiple formatting removal as you wish
- Formatting rules are stored as a stack; newer rules override older ones.
- `clear_formats` and `clear_formats_where` commands allow removing formatting globally or by condition.



(a) Part 1



(b) Part 2

Figure 4: Conditional formatting examples

5.9 Local search for cell occurrence

- We can search for the occurrences of all the commands in which a particular cell has occurred using this facility. Just type /A1 for the cell you want to search which has been previously typed.
- It will start a local search session which you can check using p,q keys

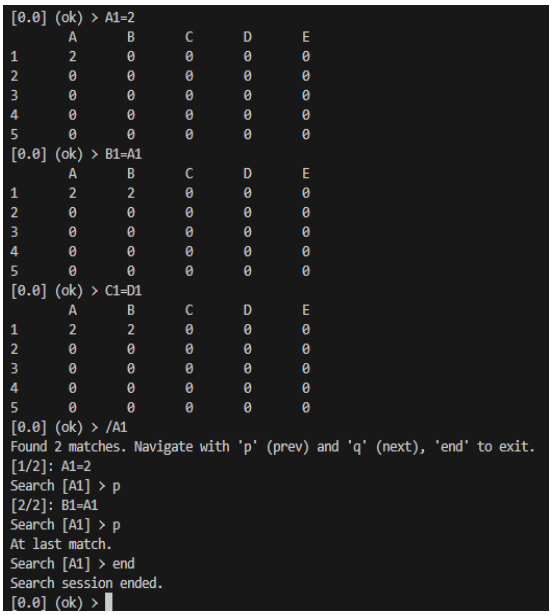


Figure 5: Cell search example

5.10 GUI

- To increase usability and readability, we have made a GUI that can be used not only to see the state of the sheet but also to edit it using the cursor. Once dependencies have been added, clicking a cell will highlight both the cells depending on it and the cells on which the selected cell depends.
- Though we were not able to show this in our demo due to time constraints, we would like you to please increase credits accordingly.
- **To open the GUI, after you type make ext1, type `gui` as the command.**
- To use the GUI to set a cell you need to double click a cell and then you can enter any formula in it like I selected cell A1 and typed B1+C1 and pressed enter. (note there is no equal to sign at the front) The same formula is displayed at the bottom in a display panel. You can set the cell value from the panel also. When we have to reset a cell value to 0 we will type 0 as the value and press enter.

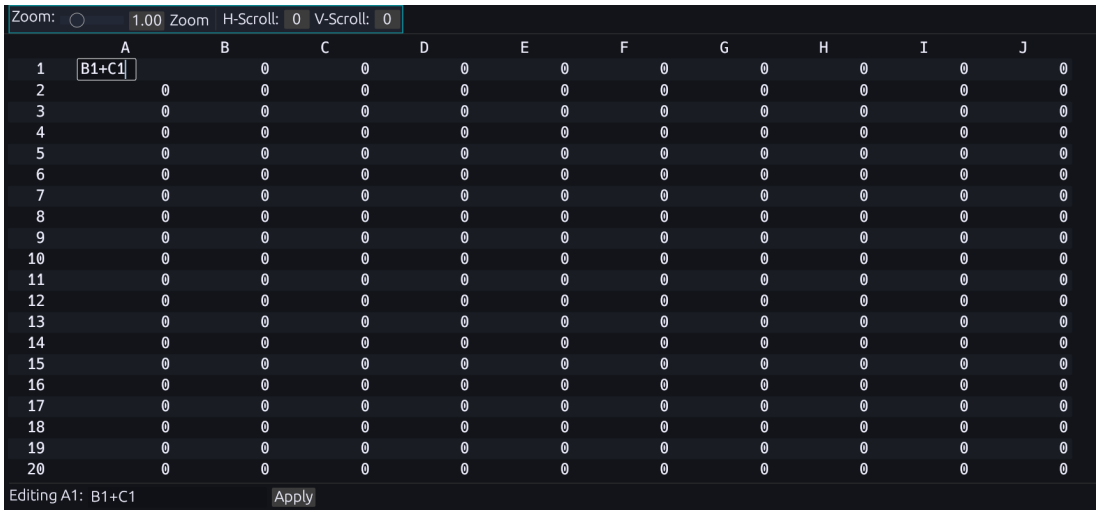
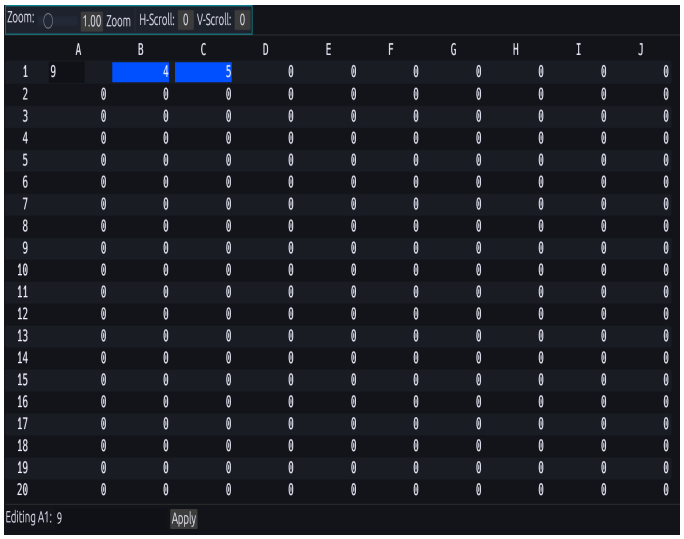
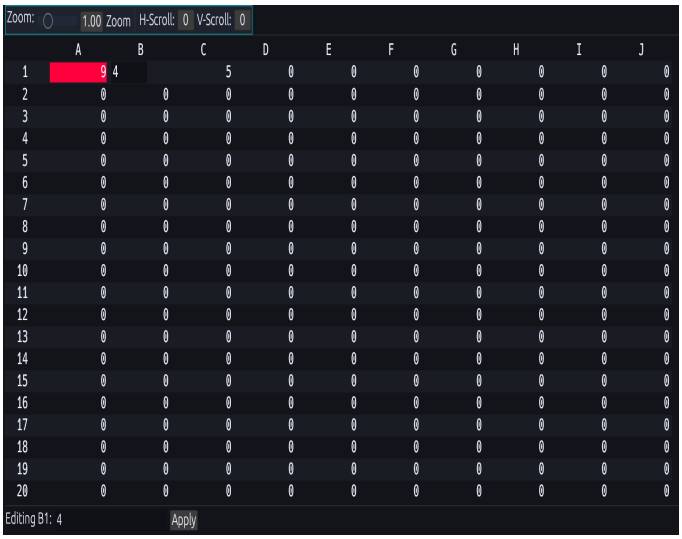


Figure 6: GUI interface

- Now when you select a cell like A1 its all parents gets highlighted and when you select B1 then its child that is A1 gets highlighted.



(a) Select A1



(b) Select B1

Figure 7: Cell dependency highlighting

- The GUI also supports **custom scrolling and zooming feature** by setting their values from the upper panel.

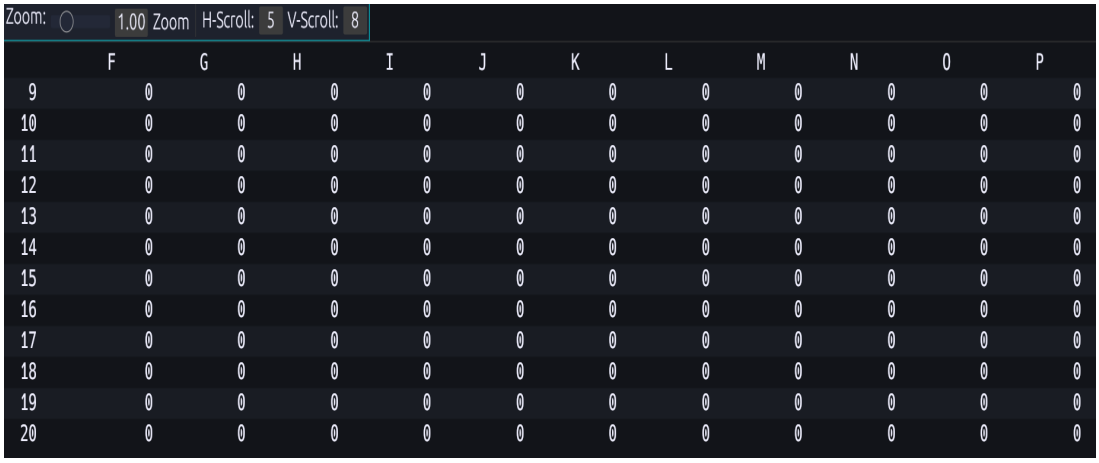


Figure 8: Scrolling and zooming controls

5.11 Plotting and Visualization

- `Plot` and `Bar` commands generate 2D plots or bar charts from cell ranges, using the `plotters` crate. **Use Plot(range) or Bar(range) to get the plots**.
- Plots are saved as PNG images in the project directory.
- The GUI, built with `eframe/egui`, displays the spreadsheet grid, supports cell selection, editing, scrolling, and highlights dependency relationships.
- A1=23
- A3=34
- Bar(A1:A4)
- Same can do for any row or column range
- Image is generated in ext1 folder bar.png
- Example walkthrough for plot
- A1=10
- B1=10
- A2=5
- B2=5
- A3=3
- B3=15
- Plot(A1:B3)
- We are making 2d plot which requires exactly 2 rows or 2 columns and then it makes a graph according to (x,y) coordinates by joining them consecutively

5.12 Privacy and Security

- Cells can be marked as private; editing them requires password entry.
- Like you type private(A1) cell is marked private next time you try to update the A1 by explicitly writing A1= any value or formula you will be prompted to enter the password which is hardcoded to **secret** and if you enter wrong password for three times the program will exit.

5.13 Input and output taking

- Input can be taken by Input(A1,input.txt) and output of the spreadsheet may be given out as Output(output.file extension). Input.txt can contain multiple lines of space separated numbers. In this way we can take input from another file directly in our program and store the output also to a desired file.

5.14 Beep sound for 3 sec if trying to access out of bounds

- scroll_to AAAA111

5.15 Itinerary Maker

- Keeping in mind the possibility of a planning trip while working on a deadline, we have an itinerary maker, which makes use of data retrieved from an API, chooses the cheapest flight path, and also adds links to hotels to stay in. Use as Flight(DEL(28/04/2025)-> (more cities the same way).....)
- flight(IXR(23/04/2025)->BOM(25/04/2025)->DEL(27/04/2025))

6 Why proposed extensions could not be implemented

- We have tried and made most of the extensions written in the proposal considering the utility of them. Some extensions we wrote happened to be very time consuming and difficult to implement considering the time boundedness but still majority of the portion has been fully covered
- The extensions proposed but not implemented were:
  - Autofill could not be done due to time constraints.
  - Cell merging, cut, copy, paste cells: could not do because we found out that we had to modify significant structure of our code which we tried.
  - Multiple sheets multiple terminals could not be done due to time issue
  - We implemented search algorithm but were not able to do search and replace as this would require extensive copying and cloning data which require significant considerations.
  - Remaining extensions we wrote happened to be redundant use cases of the preexisting ones.
  - Also we wrote about AI integration but were not able to do it due to lack of sufficient knowledge.

7 Approaches for Encapsulation

The application employs several strategies to enforce encapsulation and maintain modularity:

1. **Opaque Data Structures:** Internal state (e.g., the cell vector in `Spreadsheet`, dependency graphs in `RecalcManager`) is kept private within each module. Only specific public methods are exposed for interaction, such as `set`, `get`, `set_formula`, and `mark_private` in `Spreadsheet`.
2. **Clear Ownership:** The `Spreadsheet` and `RecalcManager` instances are owned by the main application and shared across components using `Arc<Mutex<...>>` for thread safety. All mutations are performed exclusively through their defined method interfaces.
3. **No Direct State Mutation:** Modules like `evaluator` and `main` do not access or alter internal states directly. Instead, they invoke public methods, which enforce invariants and encapsulate business logic (e.g., bounds checking in `Spreadsheet::set`, cycle detection in `RecalcManager`).
4. **Type Safety:** Communication between modules relies entirely on strongly typed enums and structs (such as `Command`, `Expr`, `CellRef`), which prevents errors due to raw or loosely typed data handling.

8 Why is this a Good Design?

This design employs several key principles that ensure robustness, maintainability, and flexibility:

1. **Separation of Concerns:** Each module has a clear, single responsibility (parsing, evaluation, data storage, dependency management, UI), which improves maintainability and testability by isolating different functionalities.
2. **Extensibility:** Adding new commands, expressions, or functions is straightforward. This can be done by extending the `Command` or `Expr` enums and updating the relevant parser and evaluator logic, without affecting unrelated modules.
3. **Type Safety and Error Handling:** The use of enums and explicit error types (e.g., `EvalError`) ensures that all potential cases are handled, minimizing runtime bugs. This also makes the code self-documenting, as each type represents a clear case.

4. **Robustness:** The recalculation module's dependency tracking and topological sorting prevent cyclic dependencies and ensure that updates occur in the correct order, which is critical for spreadsheet correctness and consistency.

5. **Encapsulation and Invariant Protection:** By exposing only high-level methods and hiding internal state, modules enforce invariants such as valid cell indices, privacy rules, and format consistency. This protection helps prevent misuse of internal data and maintains system integrity.

6. **Concurrency Safety:** Shared state, such as the `spreadsheet` and `recalculation manager`, is protected by `Arc<Mutex<...>>`, enabling safe concurrent access from both CLI and GUI threads, which is crucial for multi-threaded environments.

7. **User Interface Flexibility:** The design supports both CLI and GUI front-ends. Both interfaces interact with the core logic through the same command and evaluation mechanisms, maximizing code reuse and ensuring consistency across different user interfaces.

9 Could we implement extra extensions over and above the proposal?

- Our proposal included nearly all the good extensions though some of them we were not able to do, they have been mentioned in the above part. We had seen some of the other demos also in that also they also were either done by us or the extensions we could not do from the proposal like autofill, multiple terminal, git.

9.1 Future Improvements

- Add multi-sheet support already there in our proposal
- Add autofill already there in our proposal
- Implement JIT compilation for formulas
- Cloud synchronization capability
- Formula autocomplete in GUI

10 Whether we had to modify design

We mostly stuck to the design desicions we made from the start of the project except for a few extensions like incorporating gui we handled it separately in the main file itself so as to avoid hampering the main logic of the code and to maintain uniformity in the structure.

11 Conclusion

This Rust spreadsheet project demonstrates a robust, extensible, and interactive approach to spreadsheet computation, combining modern parsing, GUI, and concurrency techniques. It supports advanced features such as dependency-driven recalculation, conditional formatting, plotting, privacy controls, and undo/redo, debugger mode, command history, advanced formulas, for loop, cycle printing, search, input output, gui all within a modular and maintainable codebase.

12 A Kind Request

- At the time of evaluation of presentation our github repo was not structured properly and was cluttered. Considering the feedback from our TA we structured it properly, we request you to please update the marks for this section also.

- Also some part of extensions we implemented were not shown on that day due to time constraints, we request you to please pardon us and go through this documentation once as we have spent a lot of time on this and we bet our autograder code is the most optimized of all groups.

13 Links

Github Link->`https://github.com/Rndmcoder/COP290-rust-lab`